Reporte Técnico Ejecutivo de Pruebas, Evidencias y Troubleshooting — Retail 360 Enterprise

> **Proyecto Integrador Final: Arquitectura de Integración API-Led Connectivity & Agentic AI (MCP)**
> **Autor / Arquitecto de Integración:** Emmanuel Sánchez
> **Stack Tecnológico:** MuleSoft Anypoint Studio 7 (Mule Runtime 4.12.2 EE), DataWeave 2.0, PostgreSQL 16 (Docker), Salesforce CRM (OAuth 2.0 Client Credentials), MUnit 3.7.3, Model Context Protocol (MCP) + Hermes Agent (Ollama `gemma4:e4b`).
> 🎥 **Video Demostrativo End-to-End (9 minutos):** https://youtu.be/qv3NHCh6BxQ
> 📁 **Carpeta de Evidencias Fotográficas:** `evidencias/capturas/` (28 capturas PNG de alta resolución referenciadas en este documento)

📌 Nota de Uso para el Evaluador

Este documento ha sido diseñado como una **guía técnica descriptiva y autocontenida**. Para garantizar ligereza, legibilidad del código y máxima nitidez visual, las capturas de pantalla no están incrustadas dentro del cuerpo del PDF; en su lugar, **cada caso de prueba y validación referencia de forma exacta el nombre del archivo de imagen** ubicado en la carpeta adjunta `evidencias/capturas/`.

Al abrir la carpeta `evidencias/capturas/` en paralelo con este PDF, podrá inspeccionar cada evidencia en su resolución nativa (`100% zoom`), verificando tanto los payloads JSON/CSV en Postman como los registros transaccionales en PostgreSQL (Adminer) y los objetos sincronizados en Salesforce CRM.

🏛️ 1. Resumen Ejecutivo de la Arquitectura

La solución implementa el patrón **API-Led Connectivity** de MuleSoft distribuido en **5 proyectos independientes**, segregando la exposición al cliente, la orquestación de reglas de negocio, el acceso transaccional a datos y la habilitación para agentes de Inteligencia Artificial:

Capa	Proyecto MuleSoft	Puerto	Responsabilidad Principal
Experience Layer	`retail-orders-exp-api`	`8081`	Contrato público REST (RAML 1.0), propagación de `X-Correlation-ID`, negociación de contenido (`application/json` y `text/csv`) y suite de pruebas unitarias MUnit (`6/6`).
Process Layer	`retail-orders-pro-api`	`8082`	Orquestación de pedidos, cálculo de importes, enriquecimiento con catálogo, manejo de contingencia (`202 Accepted` / `PENDING_SYNC`), endpoint manual `/sync` y Scheduler autónomo cada 60 segundos.
System Layer (BD)	`retail-db-sys-api`	`8083`	Persistencia transaccional ACID (`ALWAYS_BEGIN`) sobre PostgreSQL (`retail360`), bloqueo pesimista `SELECT ... FOR UPDATE`, descuento atómico de stock e idempotencia (`409 Conflict`).
System Layer (CRM)	`retail-salesforce-sys-api`	`8084`	Integración con Salesforce Cloud CRM vía **OAuth 2.0 Client Credentials**, Upsert idempotente de cabecera (`Retail_Order__c`) y líneas (`Retail_Order_Item__c`), y detección de fallos parciales (`502 Bad Gateway`).
Agentic AI Layer (Bonus)	`retail-orders-mcp-server` + `bonus-mcp-agent`	`8085` / `STDIO`	Servidor **Model Context Protocol (MCP)** que expone herramientas de negocio (`consultar_producto`, `crear_pedido`, `consultar_pedido`, `obtener_reporte_ordenes`) consumidas en lenguaje natural por **Hermes Agent + Ollama (`gemma4:e4b`)**.

📸 2. Índice Maestro de Evidencias Fotográficas (`evidencias/capturas/`)

Todas las imágenes listadas a continuación se encuentran disponibles en la carpeta `evidencias/capturas/` :

Bloque / Caso	Nombre Exacto del Archivo en `evidencias/capturas/`	Qué Demuestra la Captura
PRE-01	`00_A_PostgreSQL_Catalogo_Semilla.png`	Contenedor Docker `retail360-postgres` activo con las tablas `products` , `retail_orders` , `retail_order_items` y el catálogo semilla inicial (`LAP-001` stock 12, `MOU-001` stock 30, `MON-001` stock 0, `CAM-001` stock 15).
PRE-02	`00_B_Anypoint_5_APIs_Deployed.png`	Consola de Anypoint Studio 7 mostrando los 5 proyectos MuleSoft (`:8081` a `:8085`) simultáneamente en estado `DEPLOYED` .
PRE-03	`00_C_Salesforce_Object_Retail_Order.png`	Salesforce Object Manager mostrando el objeto personalizado `Retail_Order__c` y su campo clave `External_Order_Id__c` (`Text(50)` , `External ID` , `Unique`).
PRE-04	`00_D_Salesforce_Object_Retail_Order_Item.png`	Salesforce Object Manager mostrando el objeto hijo `Retail_Order_Item__c` , su relación `Order__c` y su clave `External_Line_Id__c` (`Text(100)` , `External ID` , `Unique`).
CA-01	`CA01_A_Postman_Get_Product_200.png`	Petición `GET /api/v1/products/LAP-001` en Postman devolviendo `200 OK` con precio `18000` , stock `12` y `Test Results (2/2)` en verde.
CA-01	`CA01_B_PostgreSQL_Product_LAP001.png`	Registro exacto de `LAP-001` en la tabla `products` de PostgreSQL coincidiendo con el payload devuelto por la Experience API.
CA-02	`CA02_A_Postman_Create_ORD1001_201.png`	Petición `POST /api/v1/orders` para `ORD-1001` devolviendo `201 Created` , `total: 37050` , `syncStatus: "SYNCED"` , `salesforceId: "a00bm00003N2VovAAF"` y `Test Results (3/3)` .
CA-02	`CA02_B_PostgreSQL_Order_And_Stock_Deducted.png`	Verificación en PostgreSQL: `ORD-1001` persistida como `SYNCED` y stock descontado atómicamente (`LAP-001` : 12 ➔ 10, `MOU-001` : 30 ➔ 27).
CA-02	`CA02_C_Salesforce_ORD1001_Header.png`	Ficha del registro `ORD-1001` (`a00bm00003N2VovAAF`) en Salesforce Lightning con `Total__c = 37,050.00` , `Currency_Code__c = MXN` y `Status__c = SYNCED` .
CA-02	`CA02_D_Salesforce_ORD1001_Items.png`	Lista relacionada `Retail Order Items (2)` en Salesforce mostrando las líneas `ORD-1001-LAP-001` (`$36,000.00`) y `ORD-1001-MOU-001` (`$1,050.00`).
CA-03	`CA03_A_Postman_Get_ORD1001_200.png`	Petición `GET /api/v1/orders/ORD-1001` devolviendo `200 OK` con cabecera, `salesforceId` y arreglo de 2 líneas completas.
CA-04	`CA04_A_Postman_Duplicate_Order_409.png`	Reenvío de `ORD-1001` rechazado con `409 Conflict` y `error.code: "ORDER_ALREADY_EXISTS"` .
CA-04	`CA04_B_PostgreSQL_Stock_Unchanged.png`	Evidencia en PostgreSQL de que el rechazo por idempotencia no alteró el stock (`LAP-001` permanece en `10` y `MOU-001` en `27`).
CA-05	`CA05_A_Postman_Bad_Quantity_400.png`	Orden `ORD-1002` con `quantity: 0` rechazada con `400 Bad Request` (`INVALID_QUANTITY` / `VALIDATION_ERROR`).
CA-06	`CA06_A_Postman_Product_Not_Found_404.png`	Orden `ORD-1003` con SKU inexistente `XXX-999` rechazada con `404 Not Found` (`PRODUCT_NOT_FOUND`).
CA-07	`CA07_A_Postman_Insufficient_Stock_409.png`	Orden `ORD-1004` (1x `LAP-001` disponible + 1x `MON-001` agotado) rechazada con `409 Conflict` (`INSUFFICIENT_STOCK`).
CA-07	`CA07_B_PostgreSQL_Rollback_Verified.png`	Demostración de Rollback ACID en PostgreSQL: `ORD-1004` no existe en `retail_orders` y el stock de `LAP-001` se conservó intacto en `10` .
CA-08	`CA08_A_Postman_Duplicate_SKU_400.png`	Orden `ORD-1005` con dos líneas del mismo SKU `CAM-001` rechazada en Process API con `400 Bad Request` y `error.code: "DUPLICATE_SKU"` .

Bloque / Caso	Nombre Exacto del Archivo en `evidencias/capturas/`	Qué Demuestra la Captura
CA-09	`CA09_A_Postman_Contingency_202_PendingSync.png`	Con `retail-salesforce-sys-api` detenida, `POST /api/v1/orders` (`ORD-1009-CONTINGENCY`) responde `202 Accepted` con `syncStatus: "PENDING_SYNC"` .
CA-09	`CA09_B_PostgreSQL_Order_PendingSync.png`	Registro de `ORD-1009-CONTINGENCY` en PostgreSQL con `sync_status = 'PENDING_SYNC'` , `sync_attempts = 1` y stock descontado (`CAM-001` : 15 ➔ 13).
CA-10	`CA10_A_Postman_Manual_Sync_200.png`	Con `retail-salesforce-sys-api` restaurada, `POST /internal/v1/orders/ORD-1009-CONTINGENCY/sync` responde `200 OK` con `syncStatus: "SYNCED"` .
CA-10	`CA10_B_PostgreSQL_Order_Synced_NoDoubleDeduct.png`	Verificación en PostgreSQL: `ORD-1009-CONTINGENCY` actualizada a `SYNCED` con su `salesforce_id` y el stock de `CAM-001` intacto en `13` (sin doble descuento).
CA-10	`CA10_C_Salesforce_ORD1009_Synced.png`	Orden `ORD-1009-CONTINGENCY` visible y sincronizada en Salesforce CRM tras la recuperación de contingencia.
CA-11	`CA11_A_Postman_Report_CSV_200.png`	Petición `GET /api/v1/orders/report` con `Accept: text/csv` retornando `200 OK` (`Content-Type: text/csv; charset=UTF-8`) con cabecera única y detalle tabular por línea.
CA-12	`CA12_A_Partial_Error_Protection.png`	Petición `PUT http://localhost:8084/api/v1/orders/ORD-1012-PARTIAL` con fallo en la segunda línea en Salesforce devolviendo `502 Bad Gateway` (`SALESFORCE_REJECTED`).
MUNIT	`MUNIT_A_Anypoint_6_Tests_Green.png`	Panel de MUnit en Anypoint Studio con `Runs: 6/6` , `Errors: 0` , `Failures: 0` (barra 100% verde) para `retail-orders-exp-api-test-suite.xml` .
BONUS-AI	`BONUS_A_Hermes_ToolCalls_Response.png`	Terminal de Hermes Agent (`gemma4:e4b`) ejecutando tool calls MCP (`consultar_producto` , `crear_pedido`) y respondiendo en lenguaje natural con el `salesforceId` creado.
BONUS-AI	`BONUS_B_Anypoint_Console_MCP_Logs.png`	Logs de consola en Anypoint Studio evidenciando el flujo completo de las peticiones iniciadas por el agente de IA a través de las capas Experience, Process y System.

🧪 3. Desglose Técnico de Casos de Aceptación (`CA-01` a `CA-12` , `MUnit` y `Bonus AI`)

Caso `CA-01` : Consulta de Producto Existente en Catálogo

- **Objetivo:** Validar la conectividad extremo a extremo de lectura (`Experience API :8081` ➔ `Process API :8082` ➔ `DB System API :8083` ➔ `PostgreSQL :5432`).
- **Petición:** `GET http://localhost:8081/api/v1/products/LAP-001`
- **Resultado Obtenido:** `200 OK`

```
{
  "sku": "LAP-001",
  "name": "Laptop Pro 14",
  "price": 18000,
  "stock": 12,
  "active": true
}
```

- **Evidencias Asociadas:** `CA01_A_Postman_Get_Product_200.png` y `CA01_B_PostgreSQL_Product_LAP001.png` .

Caso `CA-02` : Alta de Orden Exitosa y Sincronización Inmediata con Salesforce

- **Objetivo:** Verificar el cálculo de subtotales y total en la capa Process, la inserción transaccional y descuento de inventario en PostgreSQL, y el Upsert idempotente de cabecera y líneas en Salesforce CRM vía OAuth 2.0 Client Credentials.
- **Petición:** `POST http://localhost:8081/api/v1/orders` (`orderId: "ORD-1001"` , `2x LAP-001` a `$18,000` + `3x MOU-001` a `$350`).
- **Resultado Obtenido:** `201 Created`
- Total calculado: `37050` MXN (`36000 + 1050`).
- `syncStatus` : `"SYNCED"` .
- `salesforceId` : `"a00bm00003N2VovAAF"` .
- En PostgreSQL: `LAP-001` pasó de `12` a `10` unidades; `MOU-001` pasó de `30` a `27` unidades.
- En Salesforce CRM: Se creó el registro `Retail_Order__c` (`ORD-1001`) junto con sus 2 registros hijos en `Retail_Order_Item__c` (`ORD-1001-LAP-001` y `ORD-1001-MOU-001`).
- **Evidencias Asociadas:** `CA02_A_Postman_Create_ORD1001_201.png` , `CA02_B_PostgreSQL_Order_And_Stock_Deducted.png` , `CA02_C_Salesforce_ORD1001_Header.png` y `CA02_D_Salesforce_ORD1001_Items.png` .

Caso `CA-03` : Consulta de Orden Sincronizada

- **Objetivo:** Comprobar la lectura consolidada de una orden previamente registrada, incluyendo sus líneas de detalle y metadatos de sincronización con el CRM.
- **Petición:** `GET http://localhost:8081/api/v1/orders/ORD-1001`
- **Resultado Obtenido:** `200 OK` con la estructura completa de `ORD-1001` , `syncStatus: "SYNCED"` , `salesforceId: "a00bm00003N2VovAAF"` y las 2 líneas de pedido.
- **Evidencia Asociada:** `CA03_A_Postman_Get_ORD1001_200.png` .

Caso `CA-04` : Idempotencia en Alta de Orden (`409 Conflict`)

- **Objetivo:** Garantizar que el reenvío de un `orderId` existente sea rechazado sin duplicar registros ni descontar inventario por segunda vez.
- **Petición:** `POST http://localhost:8081/api/v1/orders` con `"orderId": "ORD-1001"` .
- **Resultado Obtenido:** `409 Conflict` con código `"ORDER_ALREADY_EXISTS"` . En PostgreSQL, el stock de `LAP-001` permaneció en `10` y el de `MOU-001` en `27` .
- **Evidencias Asociadas:** `CA04_A_Postman_Duplicate_Order_409.png` y `CA04_B_PostgreSQL_Stock_Unchanged.png` .

Caso `CA-05` : Validación de Cantidad Inválida (`≤ 0`)

- **Objetivo:** Verificar el rechazo temprano de líneas con cantidades menores o iguales a cero.
- **Petición:** `POST http://localhost:8081/api/v1/orders` (`ORD-1002` , `quantity: 0`).
- **Resultado Obtenido:** `400 Bad Request` (`INVALID_QUANTITY` / `VALIDATION_ERROR`), interceptado antes de tocar la base de datos o Salesforce.
- **Evidencia Asociada:** `CA05_A_Postman_Bad_Quantity_400.png` .

Caso `CA-06` : Validación de Producto Inexistente en Catálogo (`404 Not Found`)

- **Objetivo:** Confirmar que cualquier SKU no registrado en la tabla `products` aborte la creación de la orden con estado `404` .

- **Petición:** `POST http://localhost:8081/api/v1/orders` (`ORD-1003` , SKU `"XXX-999"`).
- **Resultado Obtenido:** `404 Not Found` con `error.code: "PRODUCT_NOT_FOUND"` .
- **Evidencia Asociada:** `CA06_A_Postman_Product_Not_Found_404.png` .

Caso `CA-07` : Stock Insuficiente y Rollback Transaccional Atómico (`409 Conflict`)

- **Objetivo:** Demostrar la atomicidad ACID (`<try transactionalAction="ALWAYS_BEGIN">`) cuando una orden contiene múltiples líneas y la última falla por falta de stock.
- **Petición:** `POST http://localhost:8081/api/v1/orders` (`ORD-1004` con `1x LAP-001` [stock 10] y `1x MON-001` [stock 0]).
- **Resultado Obtenido:** `409 Conflict` (`error.code: "INSUFFICIENT_STOCK"`).
- **Verificación de Rollback:** Aunque la primera línea (`LAP-001`) sí tenía inventario, al fallar la segunda línea (`MON-001`), el bloque `On Error Propagate` revirtió toda la transacción JDBC: `ORD-1004` no fue creada y `LAP-001` conservó sus `10` unidades.
- **Evidencias Asociadas:** `CA07_A_Postman_Insufficient_Stock_409.png` y `CA07_B_PostgreSQL_Rollback_Verified.png` .

Caso `CA-08` : Validación de SKUs Duplicados en el Mismo Pedido (`400 Bad Request`)

- **Objetivo:** Impedir que una misma orden incluya dos líneas separadas con el mismo SKU (previniendo colisiones posteriores en `External_Line_Id__c`).
- **Petición:** `POST http://localhost:8081/api/v1/orders` (`ORD-1005` con dos ítems `"sku": "CAM-001"`).
- **Resultado Obtenido:** `400 Bad Request` con `error.code: "DUPLICATE_SKU"` .
- **Evidencia Asociada:** `CA08_A_Postman_Duplicate_SKU_400.png` .

Caso `CA-09` : Contingencia ante Caída de Salesforce CRM (`202 Accepted / PENDING_SYNC`)

- **Objetivo:** Demostrar el principio de continuidad de negocio: si Salesforce CRM o su System API están fuera de línea, la venta no se pierde; se confirma en PostgreSQL con inventario descontado y se encola como `PENDING_SYNC` .
- **Ejecución:** Se detuvo `retail-salesforce-sys-api` (`:8084`) y se envió `POST http://localhost:8081/api/v1/orders` con `ORD-1009-CONTINGENCY` (`2x CAM-001`).
- **Resultado Obtenido:** `202 Accepted` con `syncStatus: "PENDING_SYNC"` , `salesforceId: null` y mensaje informativo de encolamiento para reintento. En PostgreSQL, `CAM-001` descontó su stock de `15` a `13` y la orden registró `sync_attempts = 1` junto con la traza del error de conectividad en `last_sync_error` .
- **Evidencias Asociadas:** `CA09_A_Postman_Contingency_202_PendingSync.png` y `CA09_B_PostgreSQL_Order_PendingSync.png` .

Caso `CA-10` : Restauración y Sincronización Exitosa sin Doble Descuento de Stock (`200 OK / SYNCED`)

- **Objetivo:** Verificar que al restablecerse `retail-salesforce-sys-api` (`:8084`), tanto el endpoint manual `POST /internal/v1/orders/{id}/sync` como el `Scheduler` automático (cada 60s) sincronizan la orden pendiente hacia Salesforce y actualizan PostgreSQL a `SYNCED` sin volver a tocar el inventario.
- **Petición:** `POST http://localhost:8082/internal/v1/orders/ORD-1009-CONTINGENCY/sync`
- **Resultado Obtenido:** `200 OK` con `syncStatus: "SYNCED"` y `salesforceId` asignado. En PostgreSQL, el stock de `CAM-001` permaneció exactamente en `13` unidades (cero doble deducción) y `ORD-1009-CONTINGENCY` quedó registrada en Salesforce con sus líneas.
- **Evidencias Asociadas:** `CA10_A_Postman_Manual_Sync_200.png` , `CA10_B_PostgreSQL_Order_Synced_NoDoubleDeduct.png` y `CA10_C_Salesforce_ORD1009_Synced.png` .

Caso `CA-11` : Reporte Consolidado de Órdenes en Formato CSV (`200 OK`)

- **Objetivo:** Validar la transformación DataWeave a `application/csv` y negociación de contenido mediante cabecera `Accept: text/csv` .
- **Petición:** `GET http://localhost:8081/api/v1/orders/report` (`Accept: text/csv`).
- **Resultado Obtenido:** `200 OK` con `Content-Type: text/csv; charset=UTF-8` , encabezado único (`orderId,customerId,customerName,customerEmail,status,syncStatus,salesforceId,total,currency,createdAt,sku,productName,quantity,unitPrice,subtotal`) y desglose tabular línea por línea.
- **Evidencia Asociada:** `CA11_A_Postman_Report_CSV_200.png` .

Caso `CA-12` : Protección contra Error Parcial en Líneas de Salesforce (`502 Bad Gateway`)

- **Objetivo:** Comprobar que `retail-salesforce-sys-api` evalúa individualmente el resultado de cada elemento del lote devuelto por `<salesforce:upsert>` en `Retail_Order_Item__c` y nunca marca una orden como `SYNCED` si al menos una línea fue rechazada por Salesforce.
- **Petición:** `PUT http://localhost:8084/api/v1/orders/ORD-1012-PARTIAL` enviando dos líneas con el mismo SKU válido (`"LAP-001"` , 7 caracteres) para superar la validación RAML (`maxLength: 30`) y provocar una colisión de clave única `External_Line_Id__c` (`ORD-1012-PARTIAL-LAP-001`) directamente en Salesforce CRM.
- **Resultado Obtenido:** `502 Bad Gateway` con `error.code: "SALESFORCE_REJECTED"` e indicación explícita del fallo en las líneas del pedido.
- **Evidencia Asociada:** `CA12_A_Partial_Error_Protection.png` .

Suite de Pruebas Unitarias `MUnit` (`6/6` en Verde)

- **Ubicación:** `retail-orders-exp-api/src/test/munit/retail-orders-exp-api-test-suite.xml`
- **Cobertura:**

1. `get-product-by-sku-success-test` (`200 OK`)
2. `get-product-by-sku-not-found-test` (`404 Not Found`)
3. `post-orders-created-synced-test` (`201 Created`)
4. `post-orders-accepted-pending-sync-test` (`202 Accepted`)
5. `post-orders-conflict-insufficient-stock-test` (`409 Conflict`)
6. `get-orders-report-csv-test` (`200 OK` CSV)

- **Evidencia Asociada:** `MUNIT_A_Anypoint_6_Tests_Green.png` .

Bonus Diferencial: Agentic AI con Model Context Protocol (MCP) y Hermes Agent

- **Objetivo:** Exponer las capacidades de la red de aplicaciones MuleSoft como herramientas semánticas del estándar **Model Context Protocol (MCP)** para que un agente autónomo local (**Hermes Agent** con **Ollama** `gemma4:e4b`) consulte inventario, razone ante restricciones de stock y ejecute pedidos en lenguaje natural.
- **Ejecución:** El agente consultó productos en tiempo real, creó la orden `ORD-HERMES-VIDEO-01` a través de `retail-orders-exp-api` (`:8081`), descontó stock en PostgreSQL y sincronizó el pedido en Salesforce CRM.
- **Evidencias Asociadas:** `BONUS_A_Hermes_ToolCalls_Response.png` , `BONUS_B_Anypoint_Console_MCP_Logs.png` y el video demostrativo en https://youtu.be/qv3NHCh6BxQ.

🛡️ 4. Decisiones Técnicas de Arquitectura y Resiliencia

4.1. `On Error Propagate` vs `On Error Continue`

- **`On Error Propagate` (Rollback Transaccional ACID en `retail-db-sys-api`):**

Dentro de `imp-orders.xml` en `retail-db-sys-api`, el bloque `<try transactionalAction="ALWAYS_BEGIN">` agrupa la inserción de la cabecera `retail_orders` y el bucle `<foreach>` que bloquea (`SELECT ... FOR UPDATE`), descuenta (`UPDATE products`) e inserta cada línea (`INSERT INTO retail_order_items`). Su manejador de errores interno utiliza `<on-error-propagate type="ANY">`. En Mule 4, propagar una excepción hacia afuera de un ámbito `ALWAYS_BEGIN` ordena inmediatamente al *Transaction Manager* ejecutar `ROLLBACK` en PostgreSQL. Gracias a ello, en `CA-07` ninguna línea queda descontada si la última línea carece de stock.

- **`On Error Continue` (Normalización de Errores y Continuidad de Venta en Contingencia):**

1. Un bloque `<try>` externo envuelve la transacción fallida y utiliza `<on-error-continue>` para mapear el error propagado a la estructura canónica `ErrorResponse` (`400`, `404`, `409`, `502`, `503`) sin exponer stacktraces internos.
2. En `retail-orders-pro-api`, la llamada hacia Salesforce ocurre **después** de que PostgreSQL ya hizo `COMMIT` de la orden. Si `retail-salesforce-sys-api` falla por conectividad (`503`) o rechazo de negocio (`502`), el flujo captura el incidente con `<on-error-continue>`, invoca `PATCH /api/v1/orders/{orderId}/sync` para registrar `PENDING_SYNC`, `sync_attempts` y `last_sync_error`, y devuelve `202 Accepted` (`CA-09`).

4.2. Por qué se eligió `Scheduler + /sync` con Estado Persistido sobre `Until Successful`

- **Restricción crítica del negocio:** "*No se deben reintentar operaciones de alta de pedido para recuperar fallas de Salesforce*".
- Si se envolviera el flujo de creación o el conector de Salesforce dentro de un `<until-successful>` síncrono en memoria:

1. Se retendrían hilos HTTP abiertos durante minutos si Salesforce presenta una caída prolongada.
2. Si el runtime de Mule se reinicia a mitad de los reintentos, los mensajes en memoria se perderían.
3. Un reintento mal delimitado sobre el flujo de alta provocaría error `409 ORDER_ALREADY_EXISTS` o riesgo de doble descuento de inventario.

- **Solución implementada:** Desacoplamiento mediante máquina de estados en PostgreSQL (`PENDING_SYNC` ➔ `SYNCED`) gobernada por dos mecanismos complementarios e idempotentes:

1. **Reactivo / Manual (`POST /internal/v1/orders/{orderId}/sync`):** Permite forzar la sincronización bajo demanda (`CA-10`).
2. **Proactivo / Autónomo (`scheduler-retry-pending-ordersFlow`):** Se ejecuta cada **60 segundos**, consulta `GET /api/v1/orders?status=PENDING_SYNC` y reintenta únicamente el `PUT /api/v1/orders/{orderId}` hacia Salesforce usando `External_Order_Id__c` y `External_Line_Id__c`. Ambos mecanismos son 100% compatibles entre sí y jamás tocan la tabla `products`.

🛠️ 5. Matriz Maestra de Diagnóstico y Troubleshooting (22 Casos Reales de Ingeniería)

> **Valor de Ingeniería ("Oro Puro"):** Un sistema empresarial robusto no se mide únicamente porque las pruebas pasen al final, sino por el diagnóstico profundo de cada obstáculo real de infraestructura, empaquetado JVM, alcances de DataWeave, validación de esquemas RAML, seguridad OAuth 2.0 y protocolos de IA encontrado y resuelto durante su construcción.

5.1. Tabla Resumen de los 22 Incidentes Resueltos

#	Componente	Síntoma / Error Observado	Causa Raíz	Solución de Ingeniería Aplicada
1	Classpath / Config	`ResourceNotFoundException: Couldn't find resource: config${mule.env}-properties.yaml`.	Barra invertida `\` en `file="config\${mule.env}..."` escapó el signo `$` en el ClassLoader de Java.	Usar barra estándar de classpath `/`: `file="config/${mule.env}-properties.yaml"`.
2	Studio Launcher	Propiedad `${mule.env}` no reconocida al arrancar desde Anypoint Studio.	El *Tanuki Java Service Wrapper* ignora flags `-D` sin prefijo `-M-`.	Configurar `-M-Dmule.env=dev` en **Run Configurations > VM arguments**.
3	DataWeave Scope	`500 Server Error` en lugar de `503` al apagar Process API (`Unable to resolve reference of: httpStatus`).	Las variables locales `var` en `<ee:set-payload>` son invisibles dentro de `<ee:set-variable>`.	Calcular la expresión de `httpStatus` de forma autónoma dentro del script `<ee:set-variable>`.
4	HTTP Listener	Riesgo de `BindException: Address already in use` en Experience API.	El asistente gráfico creó configuraciones duplicadas (`HTTP_Listener_config`, `config1`).	Eliminar configs redundantes y unificar en `retail-orders-exp-api-httpListenerConfig` con `${api.http.port}`.
5	Studio Workspace	Proyectos invisibles en el selector múltiple de `Run Configurations`.	Los proyectos en estado *Closed Project* en Eclipse son excluidos del lanzador.	Ejecutar **Open Project** sobre los 5 proyectos en el *Package Explorer*.
6	Embedded Runtime	Diálogo bloqueante "*Already running an instance of embedded Mule runtime*".	Instancia previa de Mule activa en segundo plano reteniendo los puertos HTTP.	Confirmar **Yes** para reciclar limpiamente la JVM embebida y desplegar las APIs en paralelo.
7	MUnit Container	`PropertyNotFoundException: Couldn't find configuration property value for key ${mule.env}` en MUnit.	El contenedor embebido de MUnit corre en una JVM aislada que no hereda los *VM arguments* de Studio.	Declarar `<global-property name="mule.env" value="dev" />` como fallback en `global.xml`.
8	MUnit Matcher	Los mocks de MUnit no interceptaban el `<flow-ref>` hacia Process API.	El atributo `whereValue` apuntaba a un nombre de subflujo antiguo.	Homologar `whereValue="client_request_retailOrderProApiSub_Flow"`, logrando **6/6 tests en verde**.
9	Salesforce SOAP API	`The SOAP Login operation is not available in the API version specified (67.0)`.	El conector Salesforce v12.0.0 intenta usar por defecto la versión 67.0 en SOAP Partner Login.	Especificar explícitamente versión 60.0 o migrar a OAuth 2.0 Client Credentials.
10	Tanuki Wrapper Secrets	Argumentos `-DSFDC_PASSWORD=...` ignorados; Mule usaba el valor por defecto `placeholder`.	Ausencia del prefijo `-M-` requerido por el wrapper de Mule en Windows.	Pasar todas las propiedades de sistema con prefijo `-M-D...` en *VM arguments*.
11	Scheduler & Lockout	Cuenta de Salesforce bloqueada con `Password Lockout` en *Login History*.	El Scheduler de 60s reintentaba sincronizar órdenes `PENDING_SYNC` con credenciales inválidas en memoria.	Detener el runtime antes de rotar credenciales y desbloquear el usuario en Salesforce Setup.
12	Salesforce UI Tabs	Objeto `Retail_Order__c` creado y desplegado pero invisible en el *App Launcher*.	Los Custom Objects no aparecen en el menú Lightning sin un *Custom Object Tab* asociado.	Crear pestañas para `Retail_Order__c` y `Retail_Order_Item__c` en **Setup > Tabs**.

#	Componente	Síntoma / Error Observado	Causa Raíz	Solución de Ingeniería Aplicada
13	Salesforce SOAP Host	Error `420 Unknown: Error Page` (HTML) al enviar login SOAP al subdominio `*.develop.my.salesforce.com`.	El login SOAP Partner no autenticado solo se atiende en `login.salesforce.com`, no en *My Domain*.	Dirigir login SOAP a `login.salesforce.com` y usar *My Domain* para OAuth 2.0 (`/services/oauth2/token`).
14	Security Token Reset	Error `INVALID_LOGIN` persistente inmediatamente después de cambiar el password de Salesforce.	Cambiar la contraseña en Salesforce revoca automáticamente el Security Token anterior.	Solicitar nuevo token o eliminar dependencia de tokens mediante OAuth 2.0 Client Credentials.
15	Salesforce Setup UI	El botón `Unlock` no aparecía en la tabla general **Setup > Users**.	El botón contextual `Unlock` solo se renderiza dentro de la vista de detalle del usuario bloqueado.	Entrar al registro individual del usuario (`Sanchez, Emmanuel`) y pulsar **Unlock**.
16	OAuth 2.0 Migration	Bloqueos recurrentes y fricción operativa con SOAP Basic Authentication en orgs Winter '27.	Las integraciones servidor a servidor (M2M) modernas en Salesforce requieren OAuth 2.0 sin sesión interactiva.	Configurar **External Client App** en Salesforce y migrar `Salesforce_Config` a `<salesforce:oauth-client-credentials-connection>`.
17	MCP Payload Schema	`400 VALIDATION_ERROR` al invocar `crear_pedido` desde el servidor MCP (`:8085`).	El tool MCP recibía campos planos (`customerId`, `customerName`) pero RAML exige el objeto anidado `customer`.	Agregar transformación DataWeave en `mcp-crear-pedido-flow` que construye el objeto `customer` antes de llamar a `:8081`.
18	MCP Transport (Windows)	`CONNECT_TIMEOUT (30s)` al conectar Hermes Agent al endpoint SSE legacy (`:8085/sse`) en Windows.	Discrepancia de handshake SSE en sockets loopback IPv4/IPv6 de Windows frente a clientes modernos de IA.	Proveer el puente nativo **STDIO (`retail360_mcp_server.py`)** en paralelo al proyecto MuleSoft MCP `:8085`.
19	DataWeave Sanitización	`Salesforce rechazo la cabecera de la orden: INVALID_EMAIL_ADDRESS` en pedido creado por LLM.	El LLM incluyó asteriscos de Markdown (`emmanuel@skala.io*`) válidos en `VARCHAR` de BD pero inválidos en tipo `Email` de CRM.	Añadir función `clean(val) = trim(val as String replace '*' with '')` en `imp-orders-salesforce.xml`.
20	Propagación HTTP 202	En `CA-09`, Experience API (`:8081`) devolvía `201 Created` aunque Process API (`:8082`) retornaba `202 Accepted`.	`<http:request>` trata cualquier `2xx` como éxito y no actualizaba `vars.httpStatus` (que conservaba el default `201`).	Añadir `<set-variable variableName="httpStatus" value="#[attributes.statusCode]"/>` tras `<http:request>` en Experience API.
21	Homologación Error Code	En `CA-08`, Postman esperaba `error.code == "DUPLICATE_SKU"` pero recibía `"VALIDATION_ERROR"`.	Process API interceptaba SKUs repetidos con un código genérico distinto al definido en DB System API.	Homologar `code: "DUPLICATE_SKU"` en `retail-orders-pro-api/src/main/mule/implementation/imp-orders.xml`.
22	Simulación Fallo Parcial	En `CA-12`, enviar un SKU de 110 caracteres a `:8084` devolvía `400 Bad Request` en vez de `502 Bad Gateway`.	APIKit Router validó la restricción RAML `maxLength: 30` del SKU antes de llegar al flujo de Upsert de Salesforce.	Enviar dos líneas con el mismo SKU de 7 caracteres (`LAP-001`) para pasar APIKit y detonar la colisión de `External_Line_Id__c` en Salesforce (`502`).

5.2. Análisis Detallado de los Incidentes Clave

Caso 1 y Caso 2: Resolución de Propiedades en Classpath y Tanuki Wrapper (`-M-D`)

- **Problema:** En Windows, escribir `<configuration-properties file="config\${mule.env}-properties.yaml" />` hace que el carácter `\` escape el signo `$`, buscando literalmente `config${mule.env}-properties.yaml`. Además, pasar `-Dmule.env=dev` en Anypoint Studio no llega al runtime porque el *Tanuki Java Service Wrapper* requiere el prefijo `-M-`.
- **Solución:** Estandarizar rutas de classpath con `/` (`config/${mule.env}-properties.yaml`) y declarar todos los argumentos de la máquina virtual con `-M-D` (`-M-Dmule.env=dev`).

Caso 3: Aislamiento de Scopes entre `<ee:set-payload>` y `<ee:set-variable>` en DataWeave 2.0

- **Problema:** Cuando una API descendente estaba apagada, el bloque `<ee:transform>` del manejador de errores definía `var httpStatus` dentro del encabezado de `<ee:set-payload>` y luego intentaba referenciar `httpStatus` dentro de `<ee:set-variable variableName="httpStatus">`. Como ambos bloques se compilan como scripts DataWeave independientes, lanzaba `Unable to resolve reference of: httpStatus` (`500 Server Error`).
- **Solución:** Hacer que el script de `<ee:set-variable variableName="httpStatus">` calcule de forma autónoma el código HTTP a partir de `error.errorMessage.attributes.statusCode` y `error.errorType.identifier`.

Caso 7 y Caso 8: Aislamiento de MUnit 3.7.3 e Intercepción de Subflujos (`6/6` Verde)

- **Problema:** El contenedor embebido de Mmunit se levanta en una JVM independiente que no lee los *VM arguments* de la configuración de ejecución de Studio (`PropertyNotFoundException: ${mule.env}`). Adicionalmente, los mocks apuntaban a un nombre de subflujo desactualizado.
- **Solución:** Declarar `<global-property name="mule.env" value="dev" />` en `global.xml` (que actúa como valor por defecto sin sobrescribir variables inyectadas por línea de comandos) y alinear el atributo `whereValue="client_request_retailOrderProApiSub_Flow"` en `retail-orders-exp-api-test-suite.xml`.

Caso 9 a Caso 16: Evolución de Autenticación en Salesforce CRM hacia OAuth 2.0 Client Credentials

- **Problema:** La autenticación tradicional mediante usuario, contraseña y *Security Token* sobre SOAP Partner API presentaba tres vulnerabilidades operativas severas:

1. El conector v12 intentaba usar la versión 67.0 no habilitada en SOAP Login.
2. Cualquier cambio de contraseña invalidaba instantáneamente el *Security Token*.
3. El `Scheduler` de reintentos (cada 60s) podía bloquear la cuenta (`Password Lockout`) si quedaban órdenes en `PENDING_SYNC` durante un cambio de credenciales.

- **Solución Arquitectónica Definitiva:** Se configuró una **External Client App** en Salesforce habilitando el flujo empresarial **OAuth 2.0 Client Credentials** (`<salesforce:oauth-client-credentials-connection>`), autenticando directamente contra el endpoint `/services/oauth2/token` de *My Domain* mediante `SFDC_CLIENT_ID` y `SFDC_CLIENT_SECRET` inyectados de forma externa (Zero-Trust).

Caso 17, Caso 18 y Caso 19: Integración del Agente de IA (MCP + Hermes Agent + Sanitización CRM)

- **Problema 1 (Esquema):** El servidor MCP recibía argumentos planos del LLM pero `POST /api/v1/orders` requería un objeto `customer` anidado. Se resolvió con un `<ee:transform>` normalizador en `retail-orders-mcp-server.xml`.
- **Problema 2 (Transporte en Windows):** El transporte SSE legacy (`:8085/sse`) presentaba timeouts de handshake (`CONNECT_TIMEOUT` 30s) en sockets loopback de Windows al ser consumido por Hermes Agent. Se implementó en paralelo el adaptador **STDIO (`retail360_mcp_server.py`)**, logrando latencia de 0ms mientras se mantiene el proyecto MuleSoft `:8085` desplegado en Studio.
- **Problema 3 (Sanitización Markdown):** Durante las pruebas en vivo, el LLM incluyó un asterisco de formato Markdown en el correo (`emmanuel@skala.io`). *PostgreSQL lo aceptó en su columna* `VARCHAR`*, pero Salesforce lo rechazó en su campo tipo* `Email` *(* `INVALID_EMAIL_ADDRESS` *). Se incorporó la función DataWeave* `clean(val) = if (val ≠ null) trim(val as String replace ' ' with '') else null` en `imp-orders-salesforce.xml`, tras lo cual el `Scheduler` recuperó y sincronizó la orden automáticamente sin intervención humana.

Caso 20: Propagación del Código `202 Accepted` desde Process API hacia Experience API (`CA-09`)

- **Problema:** En `CA-09`, cuando `retail-salesforce-sys-api` estaba apagada, `retail-orders-pro-api` devolvía correctamente `202 Accepted` (`PENDING_SYNC`), pero `retail-orders-exp-api` respondía `201 Created` al cliente externo. Esto ocurría porque `<http:request>` considera cualquier código `2xx` como exitoso y no actualizaba la variable `vars.httpStatus`, dejando el valor `201` predefinido por el flujo del APIKit Router.
- **Solución:** Se añadió `<set-variable value="#[attributes.statusCode]" doc:name="Propagate HTTP Status" variableName="httpStatus" />` inmediatamente después del `<http:request>` en `client_request_retailOrderProApi.xml`, propagando fielmente tanto `201 Created` (`CA-02`) como `202 Accepted` (`CA-09`).

Caso 21: Homologación Semántica de `DUPLICATE_SKU` entre Capas (`CA-08`)

- **Problema:** La validación temprana en `retail-orders-pro-api` rechazaba pedidos con SKUs repetidos devolviendo `400 Bad Request` con `code: "VALIDATION_ERROR"`, mientras que `retail-db-sys-api` y la suite de pruebas esperaban el código específico `code: "DUPLICATE_SKU"`.
- **Solución:** Se homologó `code: "DUPLICATE_SKU"` en el bloque `Reject Duplicate SKU (400)` de `retail-orders-pro-api/src/main/mule/implementation/imp-orders.xml`.

Caso 22: Superación de Validación RAML (`maxLength: 30`) para Provocar Fallo Parcial Real en Salesforce (`CA-12`)

- **Problema:** Inicialmente, para forzar el fallo de la segunda línea en `CA-12` (`PUT http://localhost:8084/api/v1/orders/ORD-1012-PARTIAL`), se envió un SKU de 110 caracteres. Sin embargo, el APIKit Router de `retail-salesforce-sys-api` rechazó la petición en la puerta de entrada con `400 Bad Request` (`/items/1/sku expected maxLength: 30, actual: 110`) antes de invocar a Salesforce.
- **Solución:** Se diseñó un payload con dos líneas que comparten el mismo SKU válido de 7 caracteres (`"LAP-001"`). Ambas líneas cumplen el contrato RAML (`7 ≤ 30`) y pasan APIKit Router; la cabecera `Retail_Order__c` se crea en Salesforce, pero al ejecutar el Upsert del lote en `Retail_Order_Item__c`, Salesforce rechaza la segunda línea por violar la restricción `Unique` de `External_Line_Id__c` (`ORD-1012-PARTIAL-LAP-001`). El flujo de MuleSoft detecta el error parcial en `vars.itemsUpsertResult`, lanza `APP:SALESFORCE_REJECTED` y devuelve exactamente **`502 Bad Gateway`**.